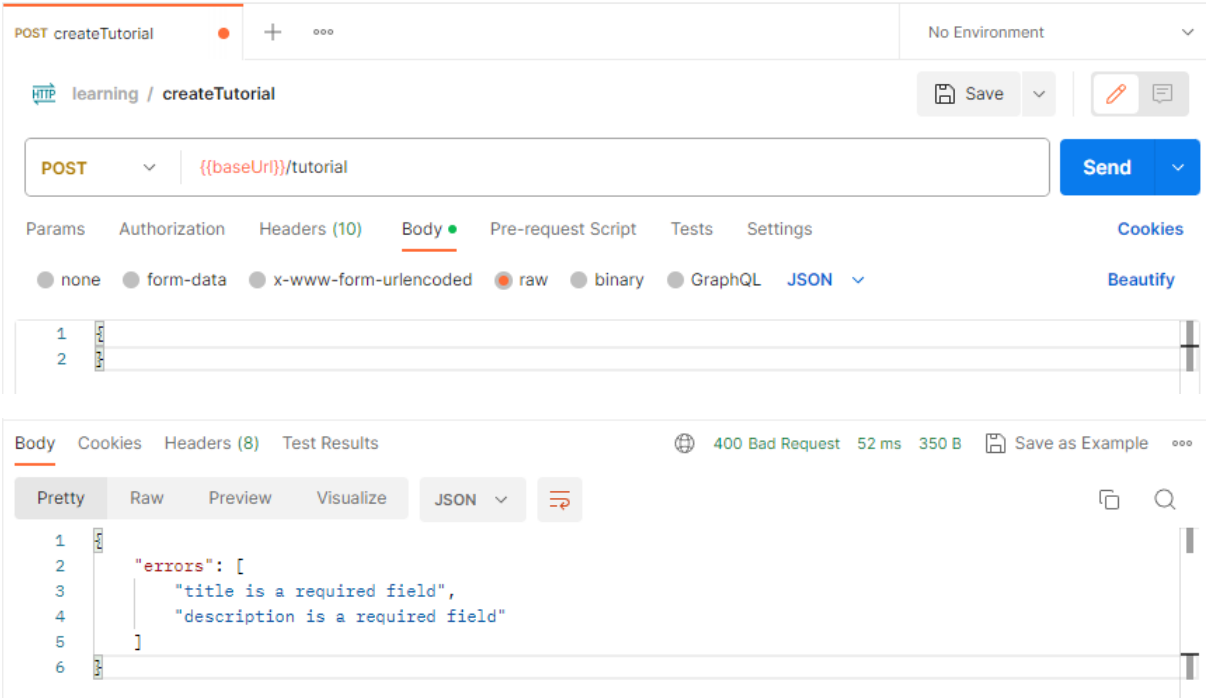
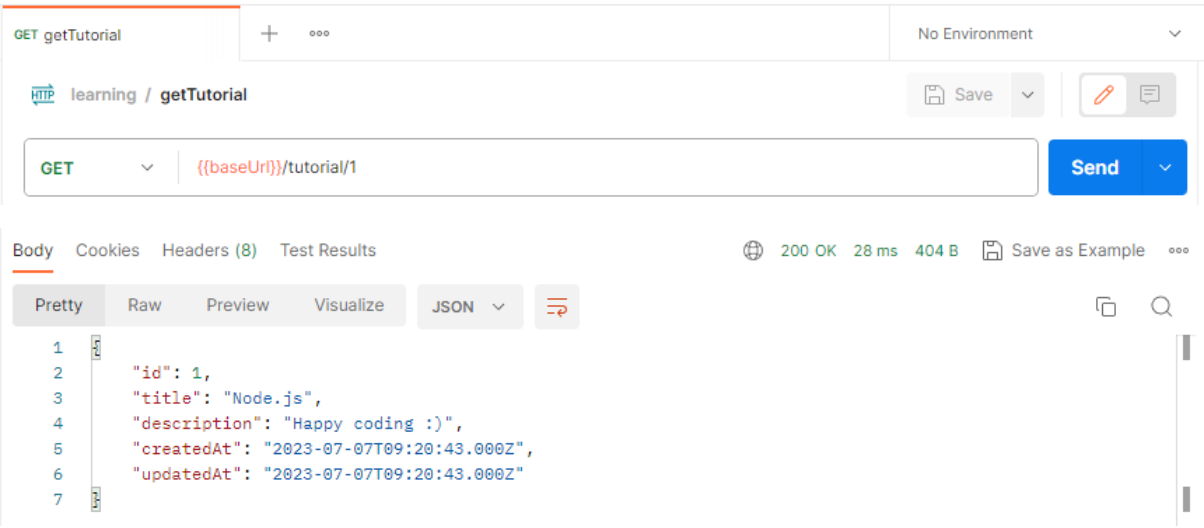
Lab 2b: Input Validation and Dynamic URL

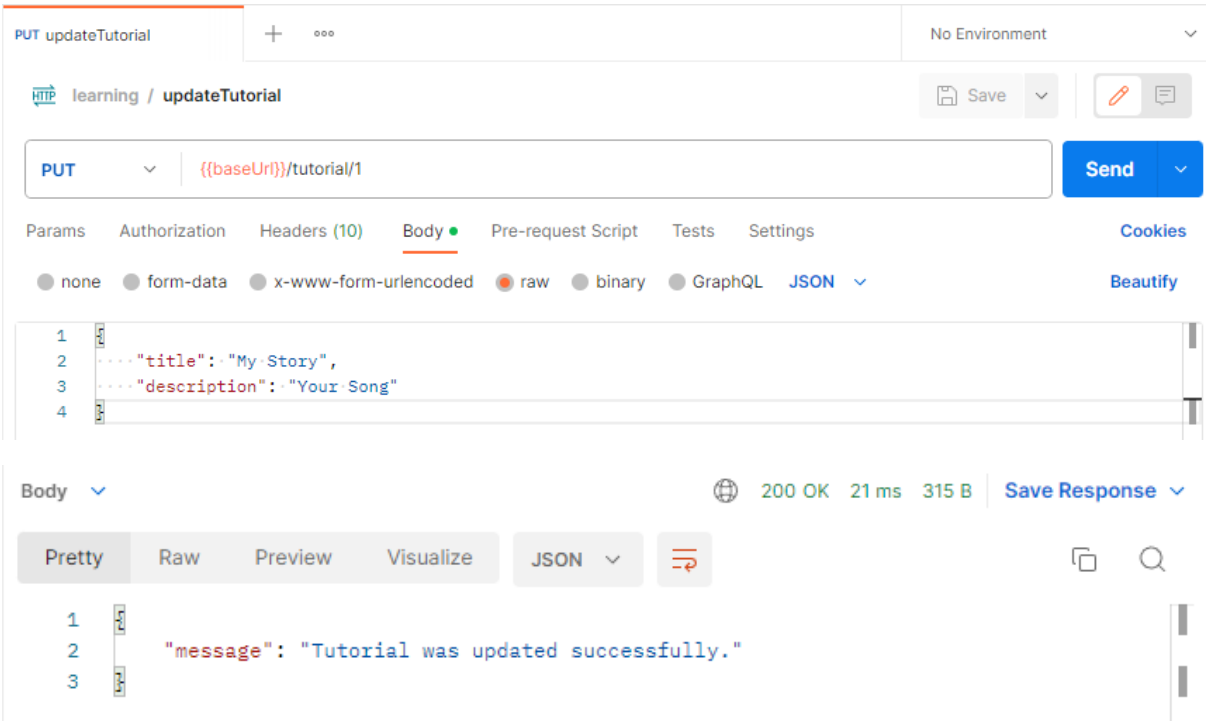
Learning Outcomes

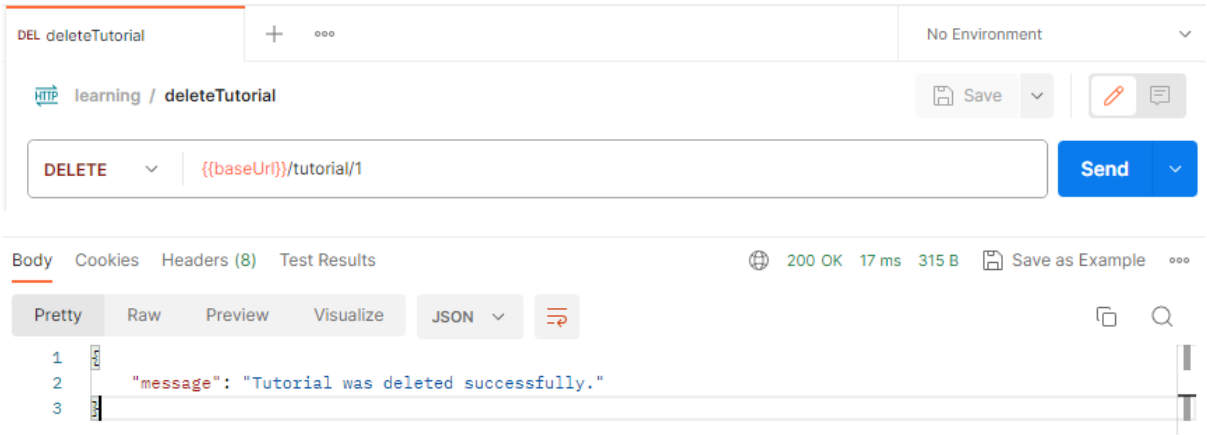
- Validate input for Post request
- Implement Get method to read data by id
- Implement Put method to update data by id
- Implement Delete method to delete data by id





Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.





Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

Activity 1: Validate input for Post request

1. Use VS Code to open the folder **learning-app**. Open terminal, change directory to **server** folder.

2. Run command to start server.

3. In Postman, remove the request body to test. Observe that the output is "could not get response".

4. In VS Code, observe the app crashed. This is because the input is not validated when title or description is null. Press Ctrl + C to stop the server.

5. Run command to install package yup:
   npm i yup
   Yup is a JavaScript schema builder for value parsing and validation.

6. Open routes/tutorial.js, add code to validate input:

```
const yup = require("yup");

router.post("/", async (req, res) => {
    let data = req.body;
    // Validate request body
    let validationSchema = yup.object({
        title: yup.string().trim().min(3).max(100).required(),
        description: yup.string().trim().min(3).max(500).required()
    });
    try {
        data = await validationSchema.validate(data,
            { abortEarly: false });
        // Process valid data
        let result = await Tutorial.create(data);
        res.json(result);
    }
    catch (err) {
        res.status(400).json({ errors: err.errors });
    }
});
```

Use yup object to define validation schema.
Call yup method validate() to validate the request body. The **validate()** method is used to validate data against the schema and returns the valid data and any applied transformations.
abortEarly: false means the validation won't stop when the first error is detected.
If the error is caught, return the bad request.

Note: **trim()** is only for string, not for other data types.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

The common used yup functions are as below:

- **Common**: required(), oneOf(array), default(value)
- **String**: string(), trim(), lowercase(), uppercase(), length(limit), min(limit), max(limit), matches(regex), email(), url()
- **Number**: number(), min(limit), max(limit), integer()
- **Date**: date(), min(limit), max(limit)
- **Boolean**: boolean()

The examples of JSON date value are as below:

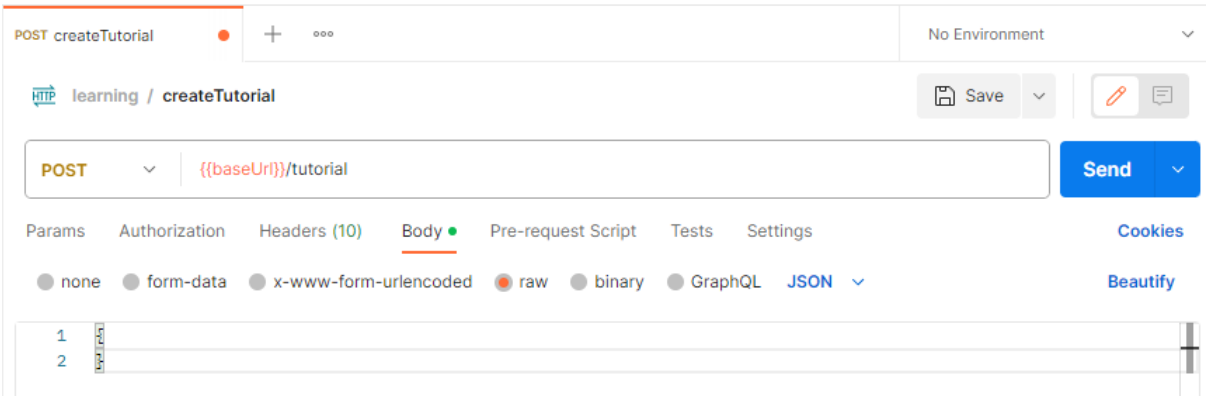
Date Only: “2020-11-20” for 20 Nov 2020

Date Time: “2020-11-20T16:50:00Z” for 20 Nov 2020 16:50:00

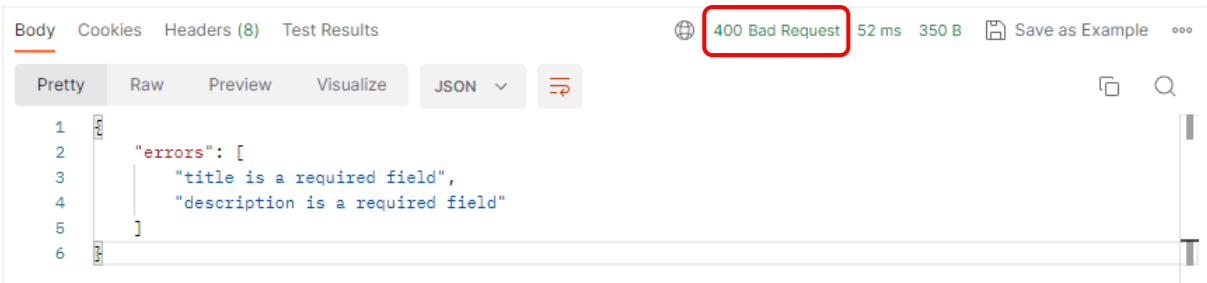
To learn more about yup API functions, can see the link below:
https://github.com/jquense/yup

7. Run command to start server.

8. In Postman, test the Post request again with an empty request body.



Observe the response status is 400 Bad Request and the errors are in the response body.



Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

Activity 2: Implement Get method to read data by id

1. In VS Code, open routes/tutorial.js, add new get function with parameter id:

```
router.get("/:id", async (req, res) => {
    let id = req.params.id;
    let tutorial = await Tutorial.findByPk(id);
    res.json(tutorial);
});
```

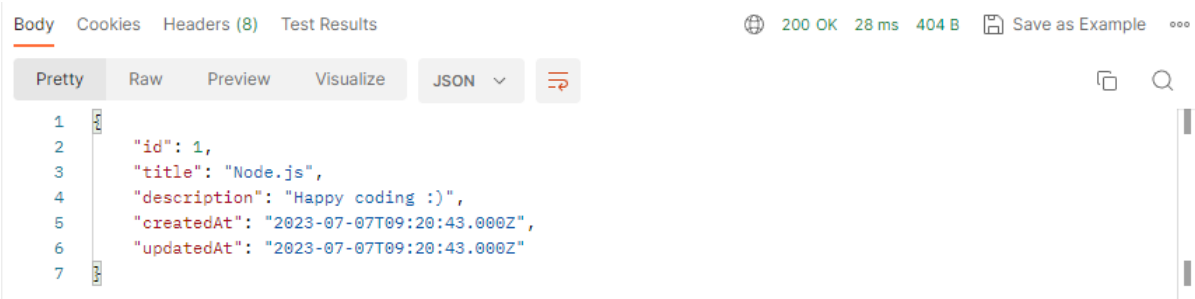
The parameter id is passed from url by using **:id**.
The sequelize method **findByPk** obtains only a single entry from the table, using the provided primary key.

2. Run the server.

3. Open Postman, add a new Get request:

Method: GET
URL: {{baseUrl}}/tutorial/<id>
Replace <id> with one id you find in the database table.

Send request and observe that the response body is the tutorial object for the found id.



4. Use one id not found in the database table and test again. The response body is **null**.

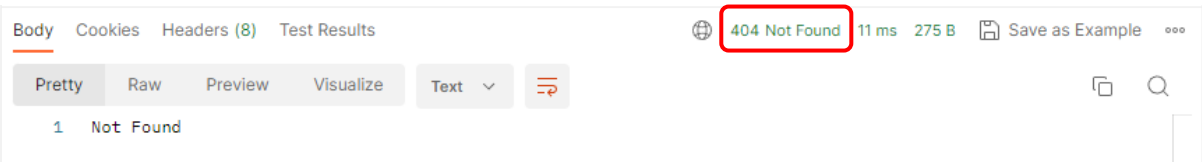
Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

5. In VS Code, add code to handle not found error:

```
    let id = req.params.id;
    let tutorial = await Tutorial.findByPk(id);
    // Check id not found
    if (!tutorial) {
        res.sendStatus(404);
        return;
    }
```

If the tutorial is null, return error code 404 for Not Found.

6. Use Postman to test the not found error. Observe the response status is 404 Not Found.



Activity 3: Implement Put method to update data by id

1. In VS Code, open routes/tutorial.js, add new put function to update tutorial by id:

```
router.put("/:id", async (req, res) => {
    let id = req.params.id;
    let data = req.body;
    let num = await Tutorial.update(data, {
        where: { id: id }
    });
    if (num == 1) {
        res.json({
            message: "Tutorial was updated successfully."
        });
    }
    else {
        res.status(400).json({
            message: `Cannot update tutorial with id ${id}.`
        });
    }
});
```

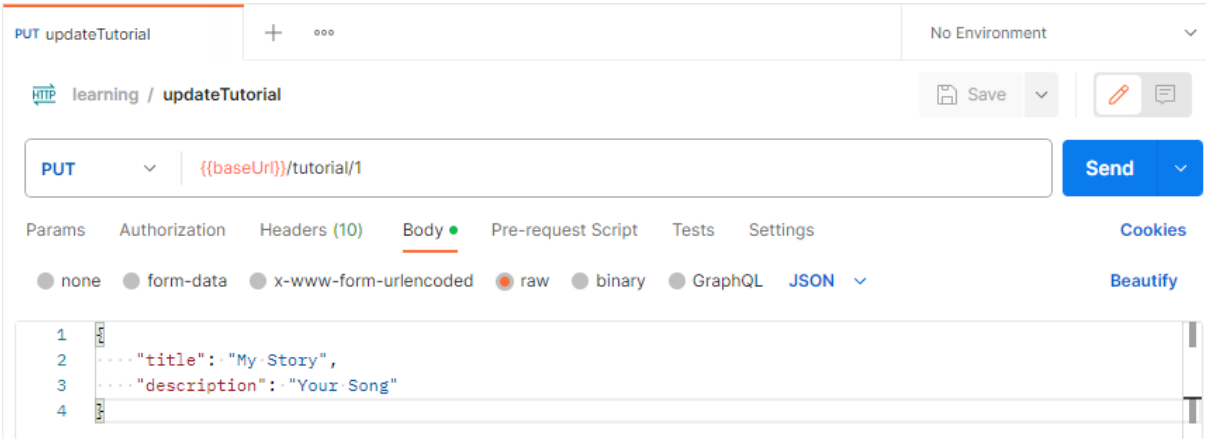
The parameter id is passed from url by using **:id**.
The updated data is from the request **body**.
The sequelize method **update()** updates data based on the where condition, and returns the number of rows affected.
If **num** equals 1, return OK, otherwise return Bad Request.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

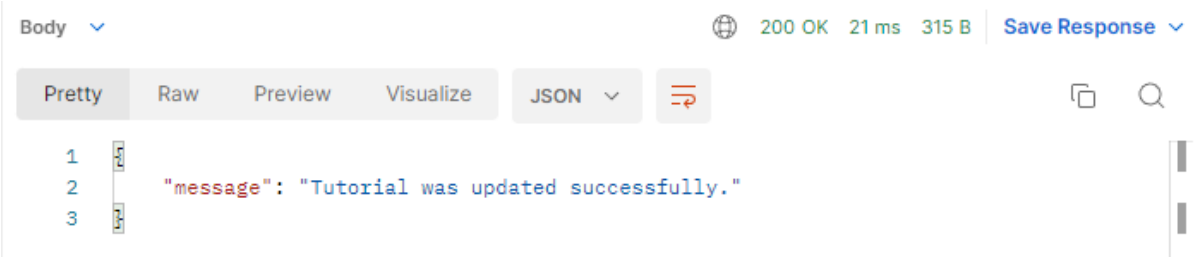
2. Open Postman, add a new Put request as below.

Method: PUT
URL: {{baseUrl}}/tutorial/<id>
Replace <id> with one id you find in the database table.
Body: (select **raw** and **JSON**)

```
{
    "title": "My Story",
    "description": "Your Song"
}
```



Send request and observe the response body.



3. In VS Code, add code to handle not found error:

```
let id = req.params.id;
// Check id not found
let tutorial = await Tutorial.findByPk(id);
if (!tutorial) {
    res.sendStatus(404);
    return;
}
```

4. Use Postman to test the not found error.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

5. In VS Code, update code to validate input:

```
    let data = req.body;
    // Validate request body
    let validationSchema = yup.object({
        title: yup.string().trim().min(3).max(100),
        description: yup.string().trim().min(3).max(500)
    });
    try {
        data = await validationSchema.validate(data,
            { abortEarly: false });

        // Process valid data
        …
    }
    catch (err) {
        res.status(400).json({ errors: err.errors });
    }
```

The validation code is almost the same as the code inside the post method.

It's a good practice to make attributes not required when performing updates in a Yup schema. This approach allows you to validate and update only the fields that are actually being modified while allowing the other fields to remain unchanged.

6. Use Postman to test invalid input.

Activity 4: Implement Delete method to delete data by id

1. In VS Code, open routes/tutorial.js, add new delete function to delete tutorial by id:

```
router.delete("/:id", async (req, res) => {
    let id = req.params.id;
    let num = await Tutorial.destroy({
        where: { id: id }
    })
    if (num == 1) {
        res.json({
            message: "Tutorial was deleted successfully."
        });
    }
    else {
        res.status(400).json({
            message: `Cannot delete tutorial with id ${id}.`
        });
    }
});
```


Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

The parameter id is passed from url by using **:id**.
The sequelize method **destroy()** deletes data based on the where condition, and returns the number of rows affected.
If **num** equals 1, return OK, otherwise return Bad Request.

2. Open Postman, add a new Delete request as below.

Method: DELETE
URL: {{baseUrl}}/tutorial/<id>
Replace <id> with one id you find in the database table.

Send request and observe the response body.

3. In VS Code, try yourself to handle the not found error.

4. Use Postman to test the not found error.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.